

Buffer Overflow Attack

Theoretical questions

Question 1: Memory structure during program execution

Answer the following sub-questions regarding memory and stack structure during program execution. Support your answers by drawing the memory and stack frames in each case.

1.1 How are the addresses decided for the following variables?

```
void func(int x, int y)
{
    int a = x + y; int b = x -
    y;
}
```

1.2 In which memory sections are the variables in the following code located?

```
int j = 0; void
foo(char *str)
{
    char *ptr = malloc(sizeof(int));
    char buffer[1024]; int i;

    static int x;
}
```

1.3 Draw the function stack frame for the following C function.

```
int func(char *str)
{
    char buf[24];
    strcpy(buf, str); return 1;
}
```

1.4 It has been suggested at times to change the way the stack grows. Instead of growing from **high address** to **low address**, it is suggested to let the stack grow from **low address** to **high address**. This way, the **buffer** will be placed above the return address, so that, in case of overflow, the **buffer** will not be able to affect the return address. Comment with arguments on the proposed change.

Question 2: Showing buffer overflow in code

Answer the following sub-questions regarding the occurrence of **buffer overflow** within programs.

2.1 In the example presented below (program **stack.c**), a **buffer overflow** occurs within the **strcpy()** function, so that the malicious code is jumped to when **strcpy()** returns, and not when **foo()** returns. Is this true or false? Justify any of your answers.

```
/* stack.c */ #include
<stdlib.h> #include <stdio.h>
#include <string.h>

int foo(char *str) {

    char buffer[24];
    strcpy(buffer, str); return 1;

}

int main(int argc, char **argv) {

    char str[517];
    FILE *badfile;
    badfile = fopen("badfile", "r"); fread(str, sizeof(char), 517,
    badfile); foo(str); printf("Returned Properly\n"); return 1;

}
```

2.2 The **buffer overflow** example in the **stack.c** program has been fixed as shown below. Is it now safe? Please justify your answer.

```
int foo(char *str, int size) {

    char *buffer = (char *) malloc(size); strcpy(buffer, str); return 1;

}
```

Question 3: Buffer overflow exploitation (attack)

Consider the following program named **exploit.c**, which is used to generate the **badfile** with which we can feed a vulnerable program. The program contains shellcode **which** when executed can start a **shell**:

```
/* exploit.c */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char shellcode[]=
    "\x31\xc0" "\x50"          /* xorl %eax,%eax */
    "\x68" //                  /* push %eax */
    "sh" "\x68" "/bin"         /* push $0x68732f2f */
    "\x89\xe3" "\x50"         /* push $0x6e69622f */
    "\x53" "\x89\xe1"         /* movl %esp,%ebx */
                                /* push %eax */
                                /* push %ebx */
                                /* movl %esp,%ecx */
    "\x99"                    /* cdq */
    "\xb0\x0b"                /* movb */          $0x0b,%al
    "\xcd\x80"                /* int */          $0x80
    ?

void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    /* Initialize buffer with 0x90 (NOP instruction) */
    memset(&buffer, 0x90, 517);

    /* You need to fill the buffer with appropriate contents here */

    /* Save the contents to the file "badfile" */
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile); fclose(badfile);
}
```

Answer the following sub-questions regarding **buffer** utilization.
overflow for carrying out attacks.

3.1 In the **exploit.c** program shown below, when assigning the value for the return address, can we do the following?

```
*((long *) (buffer + 0x24)) = buffer + 0x150;
```

Do you think the return address will show in the **shellcode** or not? Justify any of your answers.

3.2 Executing the **buffer overflow** attack in the manner described in the exercise is not always successful. Although the badfile **is** constructed correctly (with the **shellcode** at the end of the file), when we try different return addresses, the following results are observed.

```
buffer address: 0xbffff180
case 1: long retAddr = 0xbffff250 -> Able to get shell access case 2: long retAddr = 0xbffff280 -> Able to
get shell access case 3: long retAddr = 0xbffff300 -> Cannot get shell access case 4: long retAddr =
0xbffff310 -> Able to get shell access case 5: long retAddr = 0xbffff400 -> Cannot get shell access
```

Can you explain why some of these addresses work and some don't?

3.3 The following function is called within a privileged **program** (**program**). The **str** argument points to a string provided entirely by users (the string size is up to **300 bytes**). When this function is called, the address of the **buffer** array is **0xAABB0010**, while the return address is stored at **0xAABB0050**.

```
int bof(char *str)
{
    char buffer[24];
    strcpy(buffer, str); return 1;
}
```

Write down the string you would feed the program, so that when it is copied to the **buffer** and when the **bof()** function returns, the privileged program will execute your code. In your answer, you do not need to write down the code that was piped, but the offsets of the key elements in your string must be correct.

3.4 The following function is called in a **remote server** program. The argument **str** points to a **string** provided by users (the **string** size is up to **300 bytes**). The **buffer** size is **X**, which value is unknown to us (we cannot debug the **remote server** program).

```
int bof(char *str)
{
    char buffer[X];
    strcpy(buffer, str);
    return 1;
}
```

However, somehow we know the address of the **buffer array** is **0xAABBCC10**, and the distance between the end of the **buffer** and the memory location where the function's return address is located is 8. Although we don't know the exact value of X, we know that its value is between 20 and 100.

Write down the **string** that you would feed the program, so that when it is copied to the **buffer** and when the **bof() function returns**, the **remote server** program will execute your code. It is assumed that as an attacker you will only have one attempt, so you should structure the **string** in such a way that the attack will succeed even if we do not know the exact value of X. In your response, you do not need to write the code that was piped, but the distances (**offsets**) of the key elements in your string must be correct.

3.5 Why does **ASLR** (Address Space Layout Randomization) make buffer overflow attacks more difficult? Briefly describe how it works. Do the same for the **stack guard** and **non-executable stack** countermeasures .

Question 4: Heap overflow problem

In addition to stack overflow, a similar phenomenon can also occur on the heap .

4.1 Using information from various sources (online articles, books, etc.), Briefly describe how a **heap overflow** occurs and how it can be exploited to carry out an attack. What are the similarities and differences with a **buffer overflow**? In your description, it is advisable to use appropriate examples of **heap overflow attacks**.

Submit a report (theoretical part)

You will need to answer the above questions and submit the lab report.
your report in a **PDF file**.

A good understanding of the **practical part** will help you answer
and some of the theoretical questions. To answer the questions you can
use the material taught in the workshop (slides), as well as information
which you will search in other -valid- sources (online articles, books, etc.). If
If you use auxiliary sources, you must also cite the source (link or
textbook). In all questions required, you should draw the stack frame of the
each function and comment on where each element of it is stored
program.